

pyflam — Relazione Scientifica, Tecnica e Operativa

Indice

pyflam — Relazione Scientifica, Tecnica e Operativa	1
0. Sintesi esecutiva	2
1. Missione e posizionamento	2
2. Nucleo scientifico: il modello di superficie di Rothermel	3
3. Paesaggio e campo di propagazione direzionale	3
4. Crescita del fuoco: Minimum Travel Time e una nuova alternativa Eikonale	4
5. Umidità del combustibile e vento guidati dal meteo (oltre FlamMap)	4
6. Incendio di chioma: correggere la sottostima operativa	5
7. Lancio di tizzoni (spotting) dal bilancio energetico	6
8. Accoppiamento fuoco-atmosfera: piroconvezione e retroazione chioma-pennacchio-spotting	6
9. Prodotti operativi e uso in tempo quasi reale	7
10. Combustibili di chioma da dati reali (GEDI) e pipeline di chioma	7
11. Strategia di validazione	7
12. Ingegneria del software	8
13. Sintesi dei contributi innovativi oltre gli strumenti consolidati	8
Bibliografia	8
Appendice A — Formulazione fisica e matematica	10
A.1 Bilancio energetico e flussi di calore — l'incendio di superficie	10
A.2 Propagazione geometrica — Huygens, l'Eikonale e la metrica di Finsler	10
A.3 Fluidodinamica indotta dal fuoco — pennacchio di galleggiamento e vento mass-consistent	11
A.4 Fisica atmosferica — flussi verticali, convezione e geometria strato limite/LCL	11
A.5 Derivazioni dell'umidità — equilibrio, VPD, ritardo temporale e condizionamento per cella	12
A.6 Forzante WRF/GFS/ERA5 — campionamento per cella alla scala temporale	12
Dichiarazione sull'uso dell'IA	13

pyflam — Relazione Scientifica, Tecnica e Operativa

Un nuovo strumento open source, robusto e multiplatforma per la gestione, la pianificazione e la lotta agli incendi boschivi, che integra in un'unica pipeline operativa la scienza del comportamento del fuoco, dal modello di superficie di Rothermel fino alla piroconvezione fuoco-atmosfera.

Stato: passi 1–5 della roadmap implementati; passo 6 (validazione) in corso. ~8.600 righe di codice sorgente su 25 moduli; ~568 test automatici. Nucleo in puro Python (NumPy + SciPy); estensioni opzionali per dati geospaziali, atmosferici e per la compilazione JIT; OpenFOAM e Herbie individuati a runtime. Licenza MIT; integrazione continua (CI) su Python 3.11/3.12/3.13.

Versioni impaginate: [PDF](#) * [DOCX](#) * English: [PDF](#) * [DOCX](#) * [Markdown](#) (EN).

0. Sintesi esecutiva

pyflam è un nuovo strumento open source, robusto e multiplatforma, concepito per supportare la gestione, la pianificazione e la lotta agli incendi boschivi — valutazione strategica dei combustibili e del rischio, pianificazione degli interventi e supporto operativo alle decisioni. È costruito direttamente sulla scienza del fuoco pubblicata e sottoposta a revisione paritaria (Rothermel e i suoi sviluppi), ed è ispirato al paradigma operativo che FlamMap — il sistema desktop dell'USDA Forest Service / Missoula Fire Sciences Laboratory — ha definito e dimostrato utile alle agenzie antincendio di tutto il mondo: un insieme di prodotti per cella sul territorio (velocità di propagazione di superficie, intensità del fronte di fiamma, lunghezza di fiamma; potenziale di incendio di chioma; crescita del fuoco con il metodo del tempo minimo di percorrenza; probabilità di incendio da ignizioni casuali). pyflam offre quel valore operativo come software aperto, multiplatforma e programmabile (qualsiasi sistema operativo, Python, licenza MIT), interoperabile con gli stessi formati di dati della comunità (paesaggi .lcp, umidità .fms, GeoTIFF, GeoJSON) per inserirsi nei flussi di lavoro esistenti.

Su questa base pyflam va ben oltre quanto offerto dagli strumenti desktop consolidati: (1) una pipeline guidata dai dati meteorologici che ricava l'umidità del combustibile e il vento da dati di previsione/analisi in tempo reale (GFS/ERA5/WRF) e li condiziona per cella in funzione del terreno e della chioma; (2) una capacità di accoppiamento fuoco-atmosfera — solutori nativi del vento orografico, un pennacchio convettivo (CFD a galleggiamento) che il fuoco rimodella, e il lancio di tizzoni (spotting) che emerge dal bilancio energetico; (3) un modello di incendio di chioma aggiornato alla letteratura (Cruz et al. 2005/2004) che corregge la nota sottostima dello stack operativo classico Rothermel + Van Wagner; (4) un'alternativa basata sull'Eikonale anisotropa al solutore di propagazione Dijkstra su reticolo, che ne elimina gran parte del bias di griglia; e (5) diagnostiche di piroconvezione da profilo verticale (LCL, indice di Haines continuo, sounding “inverted-V”, un pennacchio di Briggs e una soglia di potenza per il pyroCb) che pilotano una retroazione del pennacchio variabile nello spazio. Il valore è duplice: pyflam è uno strumento operativo *trasparente, riproducibile e installabile*, e al tempo stesso una *piattaforma di ricerca* per l'accoppiamento fuoco-atmosfera che gli strumenti consolidati approssimano od omettono.

Il corpo del documento descrive ciascuna funzionalità e il suo fondamento scientifico; l'Appendice A fornisce la formulazione fisica e matematica completa — i bilanci di energia e di flusso, la fluidodinamica del pennacchio indotto dal fuoco e del vento orografico, la propagazione geometrica (Eikonale/Finsler), la fisica della convezione atmosferica e dei flussi verticali, e le derivazioni dell'umidità, incluso come i dati WRF/GFS/ERA5 pilotano il modello per cella e per passo temporale.

1. Missione e posizionamento

pyflam esiste per mettere una scienza del comportamento del fuoco rigorosa e aggiornata nelle mani di gestori e pianificatori come software libero, aperto e multiplatforma. I sistemi desktop consolidati (FlamMap in primo luogo) hanno dimostrato il valore operativo della modellazione del comportamento del fuoco sul territorio, ma sono binari chiusi e disponibili solo per Windows; pyflam è stato sviluppato in modo indipendente a partire dalla scienza del fuoco aperta e peer-reviewed, per offrire quel valore — e di più — come libreria programmabile, automatizzabile e installabile, eseguibile su qualsiasi sistema operativo, nel cloud o all'interno di un più ampio sistema di supporto alle decisioni.

Obiettivi operativi. La *pianificazione* strategica (valutazione dei trattamenti dei combustibili e del rischio tramite probabilità di incendio e potenziale di incendio di chioma), la *gestione* degli eventi (previsione della propagazione e del perimetro in tempo quasi reale, guidata dal meteo) e il supporto alla *lotta attiva* (dove andrà il fuoco, con quale intensità, dove lancerà tizzoni e quando potrà diventare plume-dominated / piroconvettivo). Gli stessi formati di dati della comunità sono letti e scritti, così pyflam si inserisce nei flussi di lavoro GIS e di pianificazione esistenti anziché sostituirli.

Trasparenza scientifica e validazione incrociata. Ogni modello è costruito a partire da una pubblicazione citata ed è verificato con test unitari rispetto alle equazioni pubblicate. Come controllo esterno, gli output deterministici di pyflam sono validati per confronto (diff) con raster reali di FlamMap su un paesaggio condiviso — un riferimento comodo e ampiamente affidabile, usato come *benchmark* e non come specifica da riprodurre. (Laddove lo stack consolidato è noto essere distorto — ad es. la propagazione dell'incendio di chioma — pyflam se ne discosta deliberatamente e si valida invece rispetto alla letteratura primaria; cfr. §6.)

L'insieme delle funzionalità che pyflam fornisce per l'uso operativo:

Prodotto operativo	Strumenti consolidati	pyflam
Velocità di propagazione / intensità / lunghezza di fiamma (superficie)	Sì	rothermel, landscape.basic_fire_behavior
Potenziale di incendio di chioma (superficie/passivo/attivo)	Sì	crownfire
Crescita del fuoco con Minimum Travel Time	Sì	mtt.minimum_travel_time
Probabilità di incendio da ignizioni casuali	Sì	mtt.burn_probability
I/O paesaggio .lcp / umidità .fms	Sì	io_lcp, landscape
Condizionamento dell'umidità del combustibile morto	Sì	fuel_conditioning

Tutto ciò che va oltre questa tabella — le Sezioni 5–9 — è nuova capacità che gli strumenti desktop consolidati non forniscono.

2. Nucleo scientifico: il modello di superficie di Rothermel

Il cuore scientifico del “Basic Fire Behavior” è il modello di propagazione di superficie di Rothermel (1972) con i perfezionamenti di Albini (1976): un bilancio energetico quasi-stazionario in cui la velocità di propagazione è il rapporto tra il flusso di calore ricevuto dal combustibile non bruciato e il calore necessario all'ignizione,

$$R = I_R * \xi * (1 + \varphi_w + \varphi_s) / (\rho_b * \epsilon * Q_{ig}),$$

con l'intensità di reazione I_R , il rapporto di flusso propagante ξ , i fattori di vento e pendenza φ_w , φ_s , la densità apparente ρ_b , il numero di riscaldamento efficace ϵ e il calore di pre-ignizione Q_{ig} (Rothermel 1972). pyflam implementa la formulazione completa multi-classe e multi-categoria (morto/vivo), esponendola sia come chiamata singola `spread(...)` sia come `SurfaceKernel` riutilizzabile — i termini indipendenti da vento e pendenza, calcolati una volta per combustibile + umidità e applicati a vento e pendenza scalari o per cella (array). L'intensità del fronte di fiamma segue Byram (1959): $I = H * w * R$; la lunghezza di fiamma segue $L = 0.45 * I^{0.46}$.

Gli input di combustibile usano i due insiemi standard operativi: i 13 modelli di Anderson (1982) e i 40 di Scott & Burgan (2005), incluso il trasferimento dinamico di curing della componente erbacea viva. Un fattore di carico del combustibile (scalare, per-combustibile o raster per-cella) scala i carichi standard, che sottostimano il carico reale di circa il 20–30%; poiché la risposta di Rothermel non è monotona nel carico (la lettiera compatta può rallentare oltre il rapporto di impaccamento ottimale), il fattore agisce attraverso l'intero kernel anziché scalare l'output.

Validazione. Rispetto a una run reale di FlamMap sul paesaggio toscano (1,6 milioni di celle), la ROS di superficie corrisponde entro ~3% (pendenza di regressione 0,98, r 0,9998).

3. Paesaggio e campo di propagazione direzionale

Un Landscape è lo stack di bande in memoria (modello di combustibile, pendenza, esposizione, quota, copertura/altezza/altezza di inserzione/densità della chioma). pyflam legge e scrive file .lcp FlamMap/FARSITE in puro Python (`io_lcp`, senza bisogno di GDAL) e GeoTIFF LANDFIRE/arbitrari tramite rasterio, preservando il sistema di riferimento delle coordinate per la geolocalizzazione.

Il passaggio da uno *scalare* $1 + \varphi_w + \varphi_s$ a un fuoco *direzionale* è il passo 3 della roadmap e la base della crescita del fuoco. pyflam combina il fattore di vento di Rothermel (che soffia sottovento) e il fattore di pendenza (che spinge a monte, dall'esposizione del paesaggio) come vettori, attribuendo a ciascuna cella una velocità di propagazione massima, un azimut di avanzamento (heading) e un'eccentricità dell'ellisse dal rapporto lunghezza/larghezza di Anderson (1983) (con limite superiore, come in FARSITE). La velocità direzionale rispetto all'heading ψ è

$$R(\psi) = R_{\text{max}} * (1 - e) / (1 - e \cdot \cos \psi),$$

la forma dell'ellittica (Finney 1998). Questo `SpreadField` è l'oggetto consumato da ogni solutore a valle. *Validazione*: la direzione di propagazione massima per cella corrisponde al `MAX_SPRE_DIR` di `FlamMap` entro $\sim 1^\circ$ (media $0,96^\circ$) su 1,6 milioni di celle.

4. Crescita del fuoco: Minimum Travel Time e una nuova alternativa Eikonale

4.1 Minimum Travel Time (il motore di crescita consolidato)

Il tempo di arrivo del fuoco è il percorso a tempo minimo dall'ignizione su un reticolo di direzioni di percorrenza, in cui il tempo per attraversare un segmento è la sua lunghezza divisa per la media armonica della velocità ellittica ai suoi estremi (Finney 2002). `pyflam` assembla il grafo dei tempi di percorrenza con NumPy vettorializzato (a blocchi, in CSR diretto per griglie molto grandi) e risolve il percorso minimo con il Dijkstra multi-sorgente a livello C di SciPy, scalando così a paesaggi di milioni di celle (il paesaggio toscano da 5,35 milioni di celle si risolve da una singola ignizione in $\sim 2,6$ s). `max_time` limita la ricerca per run a durata fissata.

4.2 Il limite scientifico, e la soluzione innovativa

L'MTT è *Dijkstra su un reticolo*, e un grafo offre solo le direzioni di percorrenza del suo template di vicini — perciò i tempi di arrivo portano un bias di discretizzazione angolare (di reticolo): un fuoco in assenza di vento diventa un poligono sfaccettato e gli azimut fuori reticolo risultano distorti. È la “diversa giustapposizione dei punti di calcolo” notata da Finney (2002), ed esattamente ciò che la letteratura di analisi numerica sull'equazione Eikonale anisotropa è stata costruita per eliminare. Il tempo di arrivo del fuoco soddisfa un'equazione di Hamilton–Jacobi statica; la legge di propagazione ellittica-con-vento è precisamente una metrica di Randers–Finsler (un'ellisse più una deriva), come formalizzato per il fuoco da Gahtan et al. (2026) e dalla letteratura sullo spray geodetico di Finsler.

`pyflam` offre quindi un motore di propagazione selezionabile: `method="mtt"` (predefinito) o `method="fast_marching"`, un solutore di fronte Eikonale anisotropo semi-lagrangiano che consuma lo stesso `SpreadField`. Confrontato con la soluzione analitica $distanza / R(azimut)$ su un campo uniforme, il backend Eikonale dimezza circa il bias dell'MTT su un'ellisse guidata dal vento (media $\sim 4\%$ contro $\sim 12\%$) e supera anche un template MTT più denso. Tre backend intercambiabili (JIT con Numba, con fallback NumPy) danno campi identici; il backend predefinito heap è una passata di Fast-Marching a banda stretta che pota con `max_time` come Dijkstra, seguita da una correzione Gauss–Seidel su riquadro di delimitazione — per un singolo fuoco limitato è $\sim 2x$ più veloce dell'MTT e più accurato (i percorsi su reticolo dell'MTT sono più lunghi, quindi ne sottostima l'estensione). Questo è, per quanto ci risulta, un contributo non presente negli strumenti consolidati: un solutore di propagazione Eikonale–Finsler offerto a fianco dell'MTT classico e validato rispetto a esso.

Riferimenti: Finney (2002); Sethian & Vladimirovsky (2003, Ordered Upwind Methods); Mirebeau (2014, fast-marching di Finsler); Gahtan, Shpund & Bronstein (2026, solutori Eikonali Randers–Finsler differenziabili).

4.3 Probabilità di incendio e metriche connesse

`burn_probability` riproduce l'intero insieme di output di una run MTT a ignizioni casuali di `FlamMap`: la probabilità di incendio più le metriche *connesse* — lunghezza di fiamma condizionata, intensità del fronte condizionata, le probabilità per classe di lunghezza di fiamma (il `FLP_METRIC` di `FlamMap`) e la distribuzione delle dimensioni dei singoli incendi. Oltre a `FlamMap` accetta un ensemble meteorologico (variazione del meteo fuoco-per-fuoco, ciò che rende non degenerare la distribuzione delle lunghezze di fiamma) e risolve gli incendi in chiamate Dijkstra multi-sorgente a lotti. L'intensità del fronte condizionata corrisponde alla media del `FIRE_LINE_INT` di `FlamMap` entro $\sim 2\%$ — il miglior accordo finora ottenuto su una metrica connessa.

5. Umidità del combustibile e vento guidati dal meteo (oltre FlamMap)

`FlamMap` condiziona l'umidità del combustibile morto a partire da pochi valori digitati dall'utente; `pyflam` ricava l'umidità dal meteo e la condiziona per cella.

5.1 Forzante atmosferico

atmosphere acquisisce dati di previsione/rianalisi — GFS in tempo reale tramite Herbie (dati aperti NOAA, senza autenticazione), ERA5 tramite il CDS di Copernicus, o qualsiasi colonna xarray NetCDF/GRIB — dietro un'interfaccia provider, con un ConstantAtmosphere per i test. Esso trasporta lo stato di superficie e convettivo rilevante per il fuoco (vento, T, RH, flusso di calore superficiale, CAPE, CIN, altezza dello strato limite) e ne deriva gli input di pyflam: umidità del combustibile morto all'equilibrio NFDRS (Simard 1968), vento di mezza fiamma, stabilità di Monin–Obukhov e il flusso di calore di galleggiamento ambientale. Un modello a ritardo temporale (di tipo Nelson) DeadFuelMoistureModel consente alle classi a 1/10/100 h di *ricordare* l'umidità recente anziché allinearsi istantaneamente all'ultimo valore.

5.2 Condizionamento dell'umidità del combustibile morto per cella

fuel_conditioning è l'analogo del “dead fuel moisture conditioning” di FlamMap, fondato su una specifica revisione della letteratura (Holden & Jolly 2011; Rothermel 1983; Resco de Dios 2015; Nolan 2016). I combustibili esposti al sole (versanti a sud, aperti, inclinati verso il sole) assorbono più radiazione a onde corte, risultano più caldi dell'aria e si equilibrano a un'umidità *più bassa*; i combustibili in ombra (versanti a nord, sotto chioma, di notte) restano vicini all'ambiente. Il modulo calcola la geometria solare per cella (solar_position, con correzione equazione del tempo / ora di orologio), un fattore di insolazione del terreno, un termine di trasmissione/ schermatura della chioma, e condiziona l'umidità con il sottomodulo EMC NFDRS o quello semi-meccanicistico VPD (Resco de Dios 2015 / Nolan 2016). condition_from_weather è la porta d'ingresso per l'impostazione della run: ricava l'umidità iniziale per una data/ora/posizione da un provider meteo (GFS/ERA5 in tempo reale, campionato per cella su un paesaggio geolocalizzato) oppure, in assenza di dati, da T/RH inseriti manualmente. Sul paesaggio toscano reale ciò produce una dispersione dell'umidità del combustibile fine di circa ~2x su un singolo stato meteo (ad es. 1,4–14% sotto una colonna GFS calda e secca), anziché un unico valore valido per tutto il paesaggio.

5.3 Solutori nativi del vento orografico

Il terreno reale piega il vento; un singolo valore uniforme è un'approssimazione povera. pyflam calcola un WindField su griglia in due modi, entrambi alimentano la stessa interfaccia di mezza fiamma e seguono la scienza di WindNinja (Forthofer et al.) implementata nativamente: windsolver — un solutore mass-consistent (rapido, senza dipendenze esterne); e cfd — un solutore momento/RANS tramite OpenFOAM (ingresso con strato limite atmosferico, stabilità, flussi di pendio diurni, rugosità per cella dal modello di combustibile).

6. Incendio di chioma: correggere la sottostima operativa

6.1 Il problema dello stack di FlamMap

FlamMap classifica l'incendio di chioma con l'innesco di Van Wagner (1977) + la propagazione attiva di Rothermel (1991) ($R_{active} = 3,34R_{10}$) + la classificazione superficie/passivo/attivo di Scott & Reinhardt (2001). Cruz & Alexander (2010) hanno mostrato che questo stack operativo — citando esplicitamente FlamMap, FARSITE, NEXUS, BehavePlus e FFE-FVS — presenta una significativa sottostima della propagazione dell'incendio di chioma, da tre fonti: collegamenti incompatibili tra i modelli di superficie e di chioma, la sottostima intrinseca dei modelli di ROS di Rothermel, e una riduzione non comprovata* della propagazione di chioma in funzione della frazione di chioma bruciata. La fisica dell'innesco di Van Wagner è solida; il lato della *propagazione* è superato.

6.2 L'approccio di pyflam

pyflam mantiene lo stack di FlamMap come predefinito ma rende selezionabile il modello di propagazione attiva (crown_spread="rothermel1991" | "cruz2005"). Il modello di Cruz, Alexander & Wakimoto (2005) — $CR_{OS_active} = 11,02 * U_{10}^{\frac{1}{3}} * 0,90 * CBD^{\frac{1}{3}} * 0,19 * \exp(-0,17 * M)$ (verificato rispetto alla pubblicazione originale) — è il successore validato; sul percorso Cruz un incendio di chioma attivo si propaga alla velocità *piena* di Cruz, eliminando la riduzione non comprovata per frazione di chioma. Un modello logistico di innesco dell'incendio di chioma CFIS (Cruz et al. 2004, coefficienti verificati alla fonte) fornisce una *probabilità* di passaggio in chioma come alternativa alla soglia deterministica di Van Wagner. Su un paesaggio reale con chioma derivata da GEDI (Sezione 10) il modello

di Cruz classifica come incendio di chioma attivo un numero di celle nettamente maggiore rispetto a Rothermel — la sottostima resa visibile.

Poiché FlamMap e l'intero stack Rothermel+Van Wagner *sottostimano notoriamente* l'incendio di chioma, pyflam non valida deliberatamente l'incendio di chioma rispetto a FlamMap (ciò significherebbe validare verso un riferimento distorto). La sua componente di chioma è invece *fedele ai modelli di Cruz validati dalla letteratura* (verificati con test unitari rispetto ai coefficienti pubblicati), così da ereditarne la validazione rispetto a incendi reali (Cruz 2005: ~61% della varianza su 57 osservazioni di incendi).

7. Lancio di tizzoni (spotting) dal bilancio energetico

spotting fornisce due modelli di tizzoni che si accoppiano entrambi alla crescita MTT e alla probabilità di incendio. SpottingModel è un modello rapido e parametrizzato di sollevamento-e-deriva. Quello innovativo è FirebrandPhysics: un modello stocastico e basato sulla fisica in cui lo spotting *emerge* dal sistema energetico — sollevamento da pennacchio di galleggiamento dall'intensità del fronte (Morton-Taylor-Turner), dimensione del tizzone -> velocità terminale (resistenza) -> sollevamento e burnout per combustione (legge d^2 di Tarifa), trasporto sottovento, e una probabilità di innesco all'atterraggio che diminuisce con l'umidità del combustibile ricevente. La casualità (conteggi di tizzoni di Poisson proportional to intensità, dimensioni lognormali, azimuth turbolento, innesco di Bernoulli) rende il pattern di atterraggio un esito Monte-Carlo; le costanti sono *fisiche*, legate a dati misurati di tizzoni (velocità terminali di Tohidi & Kaye e Manzello; distribuzione dimensionale di Manzello), non a calibrazioni della distanza di lancio. L'unica lunghezza debolmente vincolata è calibrata su valori di riferimento della distanza di spotting tratti dalla letteratura.

8. Accoppiamento fuoco-atmosfera: piroconvezione e retroazione chioma-pennacchio-spotting

È il contributo scientifico più distintivo di pyflam ed è assente negli strumenti consolidati.

8.1 Il pennacchio che il fuoco genera

pyroconvect ion trasforma l'intensità del fronte in un campo di flusso di calore convettivo al suolo, risolve un RANS a galleggiamento (OpenFOAM buoyantBoussinesqSimpleFoam) e restituisce il vento *incluso il pennacchio del fuoco* (correnti di richiamo/aggiornamento) — la retroazione per cui il fuoco rimodella il vento che lo guida (verificata end-to-end: una zona calda sposta il vento medio prossimo al suolo da 2,43 a 3,14 m/s). fire_atmosphere_march integra nel tempo questo accoppiamento, ricalcolando il vento di pennacchio ogni Δt minuti di crescita MTT, con il solutore del vento iniettabile così che il ciclo sia utilizzabile e testabile senza OpenFOAM.

8.2 Retroazione chioma -> pennacchio -> vento -> chioma

Il regime in cui il pennacchio conta di più è l'incendio di chioma. pyflam chiude un ciclo realmente innovativo (docs/crown_plume_coupling.md): con crown=True ogni passo della marcia ricostruisce un campo di propagazione consapevole della chioma dal vento modificato dal pennacchio corrente — le celle in chioma si propagano alla velocità di Cruz e portano l'intensità di chioma molto più elevata, che alimenta direttamente sia il solutore del pennacchio sia il lancio di tizzoni, chiudendo passaggio in chioma -> pennacchio più forte -> vento più alto -> propagazione di chioma più rapida. La retroazione positiva è limitata da una sotto-rilassamento del vento e da un tetto fisico (un test di stress sintetico conferma l'assenza di divergenza). Su dati reali l'intensità di chioma ha raggiunto circa 40x il valore di superficie, con una distanza di spotting molto maggiore.

8.3 Diagnostiche di piroconvezione da profilo verticale

Una specifica revisione su vento verticale/di deriva e sulla relazione tra strato limite e livello di condensazione ha stabilito il risultato chiave, controintuitivo: l'incendio a convezione profonda (pyroCb) è un problema verticale — uno strato limite profondo, secco e ben miscelato (LCL alto) sormontato da umidità in quota (il sounding “inverted-V”) con elevata instabilità nella bassa troposfera — e non un CAPE di superficie elevato (i pyroCb si formano regolarmente con CAPE di superficie prossimo a zero; il discriminante è l'umidità a metà troposfera). pyflam implementa

diagnostiche basate su quella geometria: `lcl_height_m`, l'indice di Haines continuo (Mills & McCaw 2010), un rilevatore di inverted-V e `pyroconvection_potential` (Castellnou et al. 2022; Peterson et al. 2017). Aggiunge un pennacchio piegato di Briggs (1969) — la forma validata del pennacchio di incendio (Lareau & Clements 2017) — e una soglia di potenza per il `pyroCb` (Tory & Kepert 2021): la potenza minima del fuoco perché il pennacchio raggiunga la condensazione contro la stabilità di sbarramento.

Queste sono integrate nella marcia accoppiata: con `pyroconvection=True` l'intensità del pennacchio è scalata da un `convective_plume_factor` consapevole del profilo, così che un'atmosfera secca/instabile inverted-V guidi un pennacchio più forte e una stabile lo smorzi — e in modalità `spatial=True` il fattore è per cella, così che sotto un'unica colonna con umidità in quota le celle localmente secche (LCL alto) ricevano il rinforzo piroconvettivo mentre quelle umide no. Ciò ribilancia l'accoppiamento convettivo verso i predittori che la letteratura sui `pyroCb` privilegia rispetto al CAPE di superficie.

9. Prodotti operativi e uso in tempo quasi reale

`pyflam` è progettato per l'operatività a supporto dell'analista, non solo per la ricerca:

- **meteo_report** — un rapporto di variazione meteo-incendio in tempo quasi reale che campiona l'atmosfera lungo la finestra della run, tracciando come variano T, RH, vento, umidità del combustibile morto (per ritardo temporale), stato convettivo (CAPE/CIN/PBL/stabilità) e flussi energetici.
- **operative** — analisi operativa di un perimetro di run: divide il perimetro in testa / fianchi / coda (o sotto-settori più fini) e scompone la spinta di propagazione nei vettori pendenza, combustibile (gradiente dell'intensità intrinseca del fronte) e vento, più la risultante e il driver dominante — le “frecce” che un front-end cartografico disegna — con export GeoJSON (forze dei settori, poligono del perimetro tracciato per contorni, riproiettato in WGS84).
- **nrt.run_realtime** — un prodotto in tempo quasi reale a chiamata singola che lega meteo -> propagazione integrata nel tempo -> perimetro -> entrambi i rapporti in un unico `RunProduct`.

Uno scenario operativo end-to-end è riproducibile tramite `tests/final_run_tuscany.py`: 300 ignizioni casuali sul paesaggio toscano reale da 5,35 milioni di celle, 28 giugno 2026, un incendio di 36 ore guidato dal meteo GFS in tempo reale con umidità condizionata per cella, che scrive ciascuna fase della pipeline nella propria cartella di output strutturata (paesaggio -> meteo -> umidità -> comportamento di superficie -> vento -> campo di propagazione -> ignizioni -> probabilità di incendio + metriche).

10. Combustibili di chioma da dati reali (GEDI) e pipeline di chioma

Non esiste un prodotto globale di altezza di inserzione / densità della chioma — GEDI e le mappe di altezza della chioma calibrate su GEDI forniscono solo l'altezza; CBH/CBD sono o US-LANDFIRE o vanno derivati. `pyflam` include un percorso riproducibile (`tests/fetch_canopy_tuscany.sh`, `tests/build_canopy_landscape_tuscany.py`) che scarica l'altezza della chioma calibrata su GEDI (Meta/WRI High-Resolution Canopy Height, Tolan et al. 2024, bucket AWS aperto), la riproietta sulla griglia toscana e deriva CBH/CBD da altezza + copertura della chioma con euristiche di scienza del fuoco trasparenti e documentate (rapporto di chioma crescente con la copertura -> base di chioma più bassa; CBD = carico scalato per copertura / profondità della chioma). Il risultato è un `.lcp` completo di chioma su cui gira l'intera pipeline di chioma — lettura -> classificazione Cruz vs Rothermel -> campo di propagazione consapevole della chioma -> marcia di chioma accoppiata al pennacchio. (Qui CBH/CBD sono *stime derivate*, chiaramente segnalate come tali, non misure di campo.)

11. Strategia di validazione

Il criterio di accettazione è un confronto cella-per-cella con output reali di `FlamMap`, non la coerenza interna. `validate` fornisce il meccanismo generico — confronto robusto di campi (bias/RMSE/rapporti/correlazione/OLS, statistiche in spazio logaritmico e “entro X%”, classificazione bruciato/non bruciato), sovrapposizione di perimetri (Jaccard/Dice/Hausdorff), accordo dei tempi di arrivo e accordo categoriale (matrice di confusione +

recall per classe, per il tipo di incendio di chioma) — con script di collegamento dei dati per ciascun paesaggio (tests/validate_flammap_*.py).

Validato finora (Toscana, 1,6 milioni di celle): ROS di superficie ~3%, direzione di propagazione massima ~1°, intensità del fronte condizionata ~2% rispetto a FlamMap. Parzialmente recuperabile: la probabilità di incendio (limitata da Monte-Carlo e dai parametri di spotting). Aperti: un confronto dell'incendio di chioma rispetto a ROS *osservate* (non FlamMap), e un confronto perimetro / tempo di arrivo di un singolo fuoco senza spotting (il dataset incluso non fornisce un raster di tempo di arrivo). I due livelli di test — test di fisica/proprietà che asseriscono relazioni note, e regressioni golden-master che fissano i valori numerici correnti — totalizzano ~568 test automatici eseguiti in CI su Python 3.11/3.12/3.13.

12. Ingegneria del software

Nucleo in puro Python su NumPy + SciPy; le estensioni opzionali abilitano le capacità più pesanti — geo (rasterio/pyproj/scikit-image), atmos (xarray/cfgrib/netcdf4/cdsapi), accel (JIT Numba per il solutore Eikonale). I motori esterni (OpenFOAM, Herbie/ERA5) sono individuati a runtime e i loro test si auto-escludono quando assenti, così il nucleo si installa ed esegue ovunque. Il design del kernel vettorializzato (un SurfaceKernel calcolato una volta e applicato a input array per cella) è ciò che rende trattabili a scala di milioni di celle il condizionamento dell'umidità su tutto il paesaggio, il forzante meteo e la classificazione della chioma. La CI è fissata (SHA-pinned) ad azioni su Node 24; la copertura è riportata a Codecov.

13. Sintesi dei contributi innovativi oltre gli strumenti consolidati

1. Umidità del combustibile guidata dal meteo, per cella — GFS/ERA5 in tempo reale -> condizionamento per insolazione del terreno + schermatura della chioma (EMC o VPD), rispetto agli scalari digitati di FlamMap.
2. Motore di propagazione Eikonale anisotropo selezionabile — un solutore Eikonale-Finsler a fianco dell'MTT, che elimina gran parte del bias di reticolo e, nella forma heap, supera Dijkstra su fuochi limitati sia in velocità sia in accuratezza.
3. Probabilità di incendio con ensemble meteorologico e l'intero set di metriche connesse.
4. Incendio di chioma aggiornato alla letteratura — propagazione attiva di Cruz 2005 + innesco logistico di Cruz 2004, che correggono la sottostima documentata dello stack operativo.
5. Accoppiamento fuoco-atmosfera — venti orografici nativi, un pennacchio CFD a galleggiamento che il fuoco rimodella, spotting basato sulla fisica, e una retroazione chioma -> pennacchio -> vento -> chioma.
6. Piroconvezione da profilo verticale — diagnostiche LCL/C-Haines/inverted-V, un pennacchio di Briggs e una soglia di potenza per il pyroCb, che pilotano una retroazione del pennacchio per cella basata sulla geometria strato limite/LCL anziché sul CAPE di superficie.
7. Livello operativo NRT — rapporti di variazione meteo e di driver del perimetro con export GeoJSON, più un prodotto in tempo reale a chiamata singola.

Il valore scientifico è che ciascuno di questi elementi è fondato sulla letteratura peer-reviewed e implementato in modo trasparente e riproducibile. Laddove un modello ben consolidato è lo standard operativo (ad es. la propagazione di superficie di Rothermel, lo stack classico di chioma) esso è mantenuto come predefinito affidabile; i metodi innovativi sono offerti come alternative validate e selezionabili — così pyflam è al tempo stesso uno strumento operativo affidabile e una piattaforma per far avanzare la scienza.

Bibliografia

(Riferimenti riportati nella lingua originale di pubblicazione.)

- Albini, F.A. (1976). *Estimating wildfire behavior and effects*. USDA FS GTR INT-30.
- Anderson, H.E. (1982). *Aids to determining fuel models for estimating fire behavior*. USDA FS GTR INT-122.
- Anderson, H.E. (1983). *Predicting wind-driven wildland fire size and shape*. USDA FS RP INT-305.

- Briggs, G.A. (1969). *Plume Rise*. USAEC TID-25075.
- Byram, G.M. (1959). *Combustion of forest fuels*. In *Forest Fire: Control and Use*.
- Castellnou, M.; Stoof, C.R.; Vilà-Guerau de Arellano, J.; et al. (2022). *Pyroconvection classification based on atmospheric vertical profiling*. J. Geophys. Res. Atmos. 127, e2022JD036920.
- Cruz, M.G.; Alexander, M.E.; Wakimoto, R.H. (2004). *Modeling the likelihood of crown fire occurrence in conifer forest stands*. Forest Science 50(5), 640–658.
- Cruz, M.G.; Alexander, M.E.; Wakimoto, R.H. (2005). *Development and testing of models for predicting crown fire rate of spread*. Can. J. For. Res. 35(7), 1626–1639.
- Cruz, M.G.; Alexander, M.E. (2010). *Assessing crown fire potential in coniferous forests of western North America: a critique of current approaches*. Int. J. Wildland Fire 19, 377–398.
- Finney, M.A. (1998). *FARSITE: Fire Area Simulator — model development and evaluation*. USDA FS RP RMRS-RP-4.
- Finney, M.A. (2002). *Fire growth using minimum travel time methods*. Can. J. For. Res. 32(8), 1420–1424.
- Forthofer, J.M.; et al. (WindNinja). *Mass-consistent and momentum diagnostic wind models for wildland fire*.
- Gahtan, B.; Shpund, J.; Bronstein, A.M. (2026). *Wildfire Simulation with Differentiable Randers–Finsler Eikonal Solvers*. arXiv:2603.00035.
- Holden, Z.A.; Jolly, W.M. (2011). *Modeling topographic influences on fuel moisture and fire danger in complex terrain*. Forest Ecology and Management 262, 2033–2041.
- Lareau, N.P.; Clements, C.B. (2017). *The Mean and Turbulent Properties of a Wildfire Convective Plume*. J. Appl. Meteorol. Climatol. 56(8).
- Manzello, S.L.; et al. *Firebrand (ember) size and generation measurements*.
- Mills, G.A.; McCaw, W.L. (2010). *Atmospheric stability environments and fire weather in Australia — the Continuous Haines index*. CAWCR Tech. Rep. 20.
- Mirebeau, J.-M. (2014). *Efficient fast marching with Finsler metrics*. Numerische Mathematik.
- Morton, B.R.; Taylor, G.; Turner, J.S. (1956). *Turbulent gravitational convection from maintained and instantaneous sources*. Proc. R. Soc. Lond. A.
- Morvan, D.; Frangieh, N. (2018). *Wildland fires behaviour: wind effect versus Byram’s convective number*. Int. J. Wildland Fire 27(10).
- Nelson, R.M. (2000). *Prediction of diurnal change in 10-h fuel stick moisture content*. Can. J. For. Res. 30, 1071–1087.
- Nolan, R.H.; Resco de Dios, V.; Boer, M.M.; et al. (2016). *Predicting dead fine fuel moisture at regional scales using vapour pressure deficit*. Remote Sensing of Environment 174, 100–108.
- Peterson, D.A.; et al. (2017). *Pyrocumulonimbus climatology — mid-troposphere humidity as a pyroCb discriminator*.
- Resco de Dios, V.; et al. (2015). *A semi-mechanistic model for predicting the moisture content of fine litter*. Agricultural and Forest Meteorology 203, 64–73.
- Rothermel, R.C. (1972). *A mathematical model for predicting fire spread in wildland fuels*. USDA FS RP INT-115.
- Rothermel, R.C. (1983). *How to predict the spread and intensity of forest and range fires*. USDA FS GTR INT-143.
- Rothermel, R.C. (1991). *Predicting behavior and size of crown fires in the Northern Rocky Mountains*. USDA FS RP INT-438.
- Scott, J.H.; Reinhardt, E.D. (2001). *Assessing crown fire potential by linking models of surface and crown fire behavior*. USDA FS RMRS-RP-29.
- Scott, J.H.; Burgan, R.E. (2005). *Standard fire behavior fuel models*. USDA FS GTR RMRS-GTR-153.
- Sethian, J.A.; Vladimirsky, A. (2003). *Ordered Upwind Methods for Static Hamilton–Jacobi Equations*. SIAM J. Numer. Anal. 41(1), 325–363.
- Simard, A.J. (1968). *The moisture content of forest fuels*. Canadian Dept. of Forestry.
- Tarifa, C.S.; et al. (1965). *On the flight paths and lifetimes of burning particles of wood*. Proc. Combustion Institute.
- Tohidi, A.; Kaye, N.B. *Aerodynamic characterization of firebrands / terminal velocity*.
- Tolan, J.; et al. (2024). *Very high resolution canopy height maps from RGB imagery (Meta/WRI HRCH)*. Remote Sensing of Environment.
- Tory, K.J.; Kepert, J.D. (2021). *Pyrocumulonimbus Firepower Threshold: Assessing the Atmospheric Potential for pyroCb*. Weather and Forecasting 36(2).
- Van Wagner, C.E. (1977). *Conditions for the start and spread of crown fire*. Can. J. For. Res. 7(1), 23–34.

Appendice A — Formulazione fisica e matematica

Le equazioni che governano ciascun livello, nella forma in cui pyflam le implementa. Notazione: g accelerazione di gravità, ρ densità, c_p calore specifico, T temperatura (K salvo diversa indicazione), θ temperatura potenziale, p pressione, U velocità del vento, u^* velocità di attrito, q'' un flusso per unità di area ($W\ m^{-2}$), I un'intensità del fronte per unità di lunghezza ($W\ m^{-1}$), R una velocità di propagazione.

A.1 Bilancio energetico e flussi di calore — l'incendio di superficie

La velocità di propagazione di Rothermel è un bilancio energetico stazionario: la velocità di propagazione è il flusso di calore ricevuto dal combustibile non bruciato diviso per il calore necessario a portarlo all'ignizione,

$$R = \frac{I_R * \xi * (1 + \varphi_w + \varphi_s)}{\rho_b * \varepsilon * Q_{ig}} \quad [m\ s^{-1} \text{ o } ft\ min^{-1}]$$

- Intensità di reazione I_R ($kW\ m^{-2}$; il tasso di rilascio di energia per unità di area del fronte fiammeggiante) = $\Gamma' * w_n * h * \eta_M * \eta_s$, con la velocità di reazione ottimale Γ' , il carico netto di combustibile w_n , il potere calorifico inferiore h e i coefficienti di smorzamento per umidità e minerali η_M , η_s . L'umidità entra *qui*, in modo non lineare, attraverso $\eta_M(M/M_x)$ — ragione per cui pyflam applica l'umidità per cella all'interno del kernel anziché all'output.
- Rapporto di flusso propagante ξ — la frazione di I_R che raggiunge il combustibile a monte, funzione del rapporto di impaccamento β e del rapporto superficie/volume σ .
- Fattori di vento e pendenza $\varphi_w = C(\beta U)^{B(\beta/\beta_{op})^{(-E)}}$, $\varphi_s = 5,275\beta(-0,3)\tan^2(pendenza)$ — *amplificazioni moltiplicative della velocità base senza vento e senza pendenza* $r_0 = I_R \xi / (\rho_b \varepsilon Q_{ig})$.
- Pozzo di calore $\rho_b \varepsilon Q_{ig}$ — densità apparente $\times$ numero di riscaldamento efficace $\varepsilon = \exp(-138/\sigma) \times$ calore di pre-ignizione $Q_{ig} = 250 + 1116 * M$ ($kJ\ kg^{-1}$), il termine che l'umidità incrementa.

La potenza del fuoco è l'intensità del fronte di Byram (1959), un flusso di energia per unità di lunghezza del fronte,

$$I = H * w * R \quad [W\ m^{-1}] \quad \text{lunghezza di fiamma } L = 0,0775 * I^{0,46} \quad [m]$$

con H resa termica e w combustibile consumato. Due vie di trasferimento del calore portano I al combustibile non bruciato: la radiazione (dominante nel fuoco plume-dominated) e la convezione (dominante quando il vento inclina la fiamma in avanti). pyflam traccia I per cella nel `SpreadField`; è la grandezza che successivamente guida il pennacchio (§A.3) e il sollevamento dei tizzoni (§7).

A.2 Propagazione geometrica — Huygens, l'Eikonale e la metrica di Finsler

Una volta che ogni cella ha una velocità di propagazione direzionale $R(\psi) = R_{max}(1-e)/(1-e*\cos\psi)$ (un'ellisse, Finney 1998), la crescita del fuoco è un problema di propagazione del fronte d'onda con due formulazioni equivalenti:

Huygens / Hamilton–Jacobi (fronte continuo). Ogni punto del perimetro è sorgente di un'ondina ellittica; il nuovo fronte è l'involuppo. Il campo del tempo di arrivo $T(x)$ soddisfa allora un'equazione di Hamilton–Jacobi statica e anisotropa (Eikonale)

$$F(x, \text{grad } T / |\text{grad } T|) * |\text{grad } T| = 1, \quad T = 0 \text{ all'ignizione},$$

dove F è la velocità normale del fronte. Quando F dipende dalla direzione (sempre, con vento e pendenza) l'equazione è *anisotropa*; per l'ellisse inclinata dal vento è *asimmetrica* (sottovento $\neq$ sopravvento) — cioè una metrica di Finsler la cui indicatrice (sfera unitaria) è l'ellisse di fuoco della cella. Il tempo di arrivo è la distanza geodetica in quella metrica.

Minimum Travel Time (grafo discreto). Si discretizza lo spazio come un grafo reticolare i cui pesi degli archi sono tempi di percorrenza $\Delta x / R(\psi)$; il tempo di arrivo è il percorso minimo (Dijkstra), Finney (2002). Esatto *come problema su grafo*, ma offre solo le direzioni discrete del template, portando quindi un errore di metricazione $O(\text{risoluzione angolare})$ — il bias di reticolo quantificato in §4.2.

Il backend `fast_marching` di pyflam risolve direttamente la forma Eikonale con un aggiornamento semi-lagrangiano: il tempo in una cella è

$$T(x) = \min \text{ sulle direzioni } [\tau(x, y) + T \text{ interpolato sul segmento per } y],$$

minimizzato su un continuo di direzioni di arrivo (non solo i nodi del reticolo), con τ il tempo di percorrenza di Finsler. Una singola passata causale ordinata a heap (l'analogo Eikonale anisotropo di Dijkstra; Sethian & Vladimirovsky 2003, Mirebeau 2014) elimina gran parte dell'errore di metricazione e permette a un orizzonte finito di potare il fronte esattamente come fa l'MTT.

A.3 Fluidodinamica indotta dal fuoco — pennacchio di galleggiamento e vento mass-consistent

Il pennacchio (momento + galleggiamento, RANS). Il fuoco immette un flusso di calore convettivo al suolo, mediato sulla cella a partire dall'intensità del fronte,

$$q''_{\text{fire}} = \chi_c * I [W m^{-1}] / \Delta x \quad [W m^{-2}] \quad (\text{pyflam: } \chi_c \sim 0,6 \text{ frazione convettiva})$$

ristretto alle celle attivamente fiammeggianti. Esso forza un sistema Reynolds-averaged Navier–Stokes stazionario a galleggiamento nell'approssimazione di Boussinesq (OpenFOAM buoyantBoussinesqSimpleFoam):

$$\begin{aligned} \text{div } u &= 0 && (\text{massa}) \\ \text{div}(u \cdot u) &= -\text{grad } p_{\text{rgh}}/\rho_0 - g*(\rho-\rho_0)/\rho_0 \hat{k} + \text{div}[(\nu+\nu_t)\text{grad } u] && (\text{momento + galleggiamento}) \\ \text{div}(u \cdot T) &= \text{div}[(\alpha+\alpha_t)\text{grad } T] + q''_{\text{fire}}/(\rho_0 c_p) && (\text{energia / calore}) \end{aligned}$$

con chiusura di turbolenza $k-\varepsilon$ e un ingresso con legge logaritmica di strato limite atmosferico. Il termine di galleggiamento $g*(\rho-\rho_0)/\rho_0$ è il motore del pennacchio: l'aria calda e leggera sopra il fuoco sale, richiamando una corrente di richiamo al suolo e un flusso di ritorno in quota. Il solutore restituisce un WindField modificato dal pennacchio che retroagisce sulla propagazione — l'accoppiamento che FlamMap omette.

Risalita del pennacchio (Briggs, piegato). Per il pennacchio di incendio piegato osservato in un flusso trasversale, la risalita scala con il flusso di galleggiamento

$$F = g*Q_c / (\pi*\rho*c_p*T) \quad [m^4 s^{-3}] ; \quad \text{sbarramento stabile: } \Delta h = 2,6*(F/(U*s))^{1/3}$$

$s = (g/\theta)*d\theta/dz$ il parametro di stabilità statica (Brunt–Väisälä al quadrato). Invertendo Δh per Q_c si ottiene la soglia di potenza per il pyroCb (§8.3): la minima potenza del fuoco che solleva il pennacchio alla condensazione contro lo sbarramento.

Vento orografico mass-consistent (diagnostico, senza momento). Dove il RANS completo è troppo oneroso, pyflam usa il metodo variazionale di Sasaki (1970): trovare il vento più vicino a una prima stima interpolata u_0 che sia a divergenza nulla. Con un moltiplicatore di Lagrange λ ciò si riduce a un problema di Poisson anisotropo,

$$d^2\lambda/dx^2 + d^2\lambda/dy^2 + T_R*d^2\lambda/dz^2 = -2 \text{div}(u_0) , \quad u = u_0 + 1/2 \text{grad}_h \lambda , \quad w = w_0 + (T_R/2)*\lambda_z$$

(T_R il rapporto di stabilità/anisotropia) — accelerazione di cresta, incanalamento di valle e decelerazione sottovento emergono dalla sola conservazione della massa (Forthofer et al. 2014).

A.4 Fisica atmosferica — flussi verticali, convezione e geometria strato limite/LCL

Flussi di strato superficiale (similarità di Monin–Obukhov). Lo scambio turbolento proprio dell'atmosfera fissa lo sfondo in cui il pennacchio si sviluppa. Dal flusso di calore sensibile superficiale q''_H e dalla velocità di attrito $u^* = \kappa U / \ln((z+z_0)/z_0)$,

$$L = -u^{*3} \rho c_p T / (\kappa g q''_H) \quad (\text{lunghezza di Obukhov})$$

$L < 0$ instabile (convettivo diurno, il pennacchio cresce liberamente), $L > 0$ stabile (sbarrato). pyflam classifica la stabilità dal segno di q''_H (con un override CAPE/CIN) e smorza o rinforza di conseguenza il fattore convettivo del pennacchio.

Convezione umida e profilo verticale. Se il pennacchio si limita a salire o si approfondisce in pirocumulo/pirocumulonembo dipende dalla *struttura termodinamica verticale*, non dal galleggiamento di superficie. Grandezze chiave, tutte calcolate da una colonna di profilo:

- Temperatura potenziale $\theta = T*(1000/p)^{0,286}$ — conservata nell'ascesa adiabatica secca; uno strato limite ben miscelato ha $\theta \sim \text{costante}$ ($s \sim 0$), così il pennacchio sale indisturbato fino al livello di condensazione.
- Livello di condensazione per sollevamento LCL — la quota a cui una particella di superficie va sollevata per saturarsi; pyflam usa $LCL \sim 125*(T - T_d) \text{ m}$ (Espy/Lawrence), con il punto di rugiada T_d dalla relazione di Magnus inversa. Uno strato miscelato caldo e secco $\Rightarrow$ grande deficit di rugiada $\Rightarrow$ LCL alto.

- CAPE / CIN — l'energia di galleggiamento rilasciata / l'inibizione da superare. È fondamentale che i pyroCb si formino regolarmente con CAPE di superficie prossimo a zero: nel sounding canonico “inverted-V” (strato miscelato profondo e secco sormontato da umidità in quota) le particelle che contano provengono dallo strato caldo e secco e trovano umidità *sopra* l'LCL. Il discriminante è quindi l'umidità a metà troposfera, non il CAPE di superficie.
- Haines continuo C-Haines = CA(stabilità, gradiente 850->700 hPa) + CB(secchezza, deficit di rugiada a 700 hPa) — l'indice operativo di instabilità+secchezza della bassa troposfera.

pyroconvection_potential di pyflam segnala una colonna come favorevole al pennacchio quando lo strato limite è profondo e secco (LCL alto) e vi è umidità/instabilità in quota (inverted-V o C-Haines elevato) — la geometria strato limite->LCL->convezione libera identificata dalla letteratura sui pyroCb (Castellnou 2022; Peterson 2017; Tory & Kepert 2021).

A.5 Derivazioni dell'umidità — equilibrio, VPD, ritardo temporale e condizionamento per cella

Umidità all'equilibrio (istantanea). L'umidità a cui un combustibile morto tende in aria costante è l'EMC NFDRS a tratti $m_e(T, RH)$ (Simard 1968), crescente con la RH e debolmente decrescente con T. L'alternativa semi-meccanicistica lega l'umidità del combustibile fine al deficit di pressione di vapore, $M = a + b \exp(-cVPD)$ (Resco de Dios 2015 / Nolan 2016 Eq. 8: $a=7,86$, $b=140,94$, $c=3,73$; VPD in kPa), con $VPD = e_s(T)(1 - RH/100)$.

Dinamica a ritardo temporale (memoria nel tempo). I combustibili reali ritardano rispetto all'aria; ciascuna classe dimensionale si rilassa verso l'EMC secondo un'EDO del primo ordine

$$dm/dt = (m_e - m)/\tau \Rightarrow m(t+\Delta t) = m_e + (m(t) - m_e) * e^{(-\Delta t/\tau)}$$

con $\tau = 1, 10, 100$ h. Il DeadFuelMoistureModel di pyflam integra questa nel tempo attraverso i passi della marcia, così che le umidità a 1/10/100 h conservino la memoria dell'umidità recente anziché allinearsi all'ultimo valore — essenziale per l'essiccamento diurno e le run di rianalisi pluriorarie.

Condizionamento per cella di terreno/chioma. La stessa aria produce un'umidità del combustibile molto diversa su un versante soleggiato a sud e in un impluvio in ombra. pyflam calcola, per cella, un indice di esposizione solare S in $[0,1] = (\text{fattore di radiazione diretta sulla faccetta del versante}) \times (\text{apertura della chioma})$:

$$\cos(\text{incidenza}) = \cos(\text{pendenza}) \cos(\text{zenit}) + \sin(\text{pendenza}) \sin(\text{zenit}) \cos(\text{az_sole} - \text{esposizione})$$

$$S = \max(\cos \text{ incidenza}, 0) * (1 - \text{schermatura} * \text{copertura_chioma})$$

con zenit/azimut solare da latitudine, giorno dell'anno e ora (corretta per l'equazione del tempo). Viene aggiunto un riscaldamento prossimo al combustibile $\Delta T = \Delta T_{\text{sun}} S$; *mantenendo costante la pressione di vapore superficiale, il combustibile più caldo vede una RH locale più bassa* $RH = 100 e_{\text{vap}}/e_s(T+\Delta T)$, e l'EMC (o il VPD) è valutato in quel microclima più caldo e più secco. Le celle esposte al sole si seccano quindi al di sotto dell'ambiente, quelle in ombra restano vicine ad esso — una dispersione dell'umidità del combustibile fine di circa $\sim 2x$ su un singolo paesaggio sotto un identico stato meteo (Holden & Jolly 2011; Rothermel 1983).

A.6 Forzante WRF/GFS/ERA5 — campionamento per cella alla scala temporale

pyflam consuma dati di previsione numerica e di rianalisi attraverso un'unica interfaccia provider, con tre scale di risoluzione accoppiate:

- Per cella (spaziale). `field_on(ls, time)` campiona la colonna su griglia su ogni cella del paesaggio (vicino più prossimo nella griglia sorgente, longitudini riavvolte alla convenzione sorgente — GFS 0–360°, ERA5 –180–180°), così che vento, temperatura, umidità e l'umidità del combustibile morto derivata varino sul dominio. Su un paesaggio geolocalizzato le latitudini/longitudini dei centri cella sono riproiettate dal CRS del paesaggio (ad es. ETRS89-LAEA) a coordinate geografiche.
- Per passo temporale (temporale). `fire_atmosphere_march` rilegge il provider all'ora di orologio avanzante ogni Δt minuti, così che propagazione, pennacchio e umidità rispondano al meteo in evoluzione — una previsione (ora di previsione GFS fxx) o una rianalisi (ERA5 oraria) — e l'umidità a ritardo temporale (§A.5) conservi lo stato tra i passi.
- Riconciliazione delle unità di flusso. I flussi superficiali ERA5 sono *accumulati* ($J m^{-2}$ sul passo del prodotto) e positivi verso il *basso*; pyflam li converte in $W m^{-2}$ istantanei positivi verso l'*alto* ($q'' = -J m^{-2} / \Delta t_{\text{accum}}$),

così che segno e unità corrispondano allo sfondo a galleggiamento di §A.3/A.4. I flussi istantanei WRF/GFS passano direttamente.

In concreto, la run toscana end-to-end (§9) preleva una colonna GFS in tempo reale per il 28 giugno 2026, ricava $T=36,5\text{ °C}$ / $RH=25\%$ al sito, condiziona l'umidità del combustibile morto per cella a 1,4–14% su 5,35 milioni di celle per insolazione del terreno, e la alimenta — con il vento GFS e, opzionalmente, il rinforzo del pennacchio inverted-V (§8.3) — ai solutori di propagazione MTT/Eikonale e di probabilità di incendio. Ogni livello fisico sopra descritto è esercitato in quella singola run, su dati reali, alla risoluzione nativa del paesaggio.

Dichiarazione sull'uso dell'IA

Questa relazione, e parti sostanziali dell'implementazione di pyflam che essa descrive, sono state prodotte con l'assistenza di uno strumento di codifica/ricerca basato sull'IA (Claude). I modelli di scienza del fuoco sottostanti sono implementazioni indipendenti delle pubblicazioni peer-reviewed citate; le revisioni della letteratura che hanno motivato le componenti innovative sono state condotte con ricerca assistita dall'IA, e ogni fonte citata è stata reperita tramite ricerche dal vivo e (ove centrale) verificata alla fonte primaria. I coefficienti dei modelli empirici sono stati confrontati con le pubblicazioni originali prima dell'implementazione.